

Laboratoire 1

Contraintes et Declencheurs SQL

Base de donnees Hopital

Genere le 08/05/2025

Table des matieres

1. Introduction aux contraintes et declencheurs...	3
1.1. Types de contraintes SQL.....	3
1.2. Types de declencheurs.....	4
1.3. Importance dans une base de donnees hospitaliere	4
2. Contraintes applicatives.....	5
2.1. Contrainte sur le sexe.....	5
2.2. Contrainte sur l'etat civil.....	6
2.3. Contrainte sur le groupe sanguin.....	6
2.4. Contrainte sur le compte bancaire.....	7
2.5. Contraintes d'integrite referentielle.....	8
3. Declencheurs (Triggers).....	9
3.1. PatientsInsertNrSis.....	10
3.2. HopitauxServicesNbreMed.....	11
3.3. PatientsUpdateNrSis.....	12
3.4. PatientsDelNrSIS.....	13
4. Tests et validation.....	14
4.1. Tests des contraintes.....	14
4.2. Tests des declencheurs.....	15
4.3. Scenarios de test avances.....	16
5. Bonnes pratiques et optimisation.....	17
6. Conclusion.....	18

Laboratoire 1 - Contraintes et Declencheurs SQL

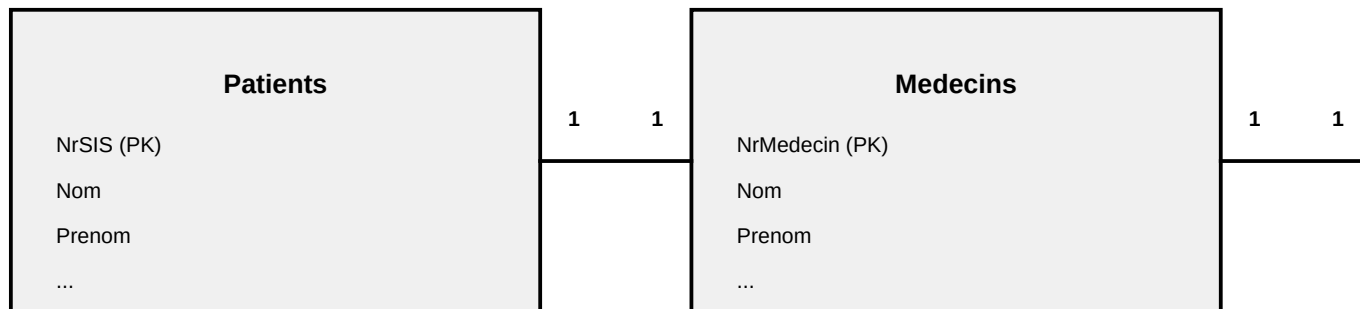
1. Introduction aux contraintes et declencheurs

Les contraintes et les declencheurs sont des mecanismes essentiels dans les systemes de gestion de bases de donnees relationnelles (SGBDR) qui permettent de maintenir l'integrite et la coherence des donnees.

Les contraintes sont des regles qui definissent les valeurs acceptables pour les donnees. Elles sont verifiees automatiquement par le SGBDR lors de l'insertion, la mise a jour ou la suppression de donnees.

Les declencheurs (triggers) sont des procedures stockees qui s'executent automatiquement en reponse a certains evenements sur une table, comme l'insertion, la mise a jour ou la suppression de donnees. Ils permettent d'implementer des regles metier complexes et d'automatiser certaines taches.

Schema simplifie de la base de donnees Hopital



1.1. Types de contraintes SQL

SQL offre plusieurs types de contraintes pour garantir l'integrite des donnees:

1. NOT NULL: Garantit qu'une colonne ne peut pas avoir de valeur NULL.
2. UNIQUE: Assure que toutes les valeurs d'une colonne sont differentes.
3. PRIMARY KEY: Combine NOT NULL et UNIQUE, identifie de maniere unique chaque ligne.
4. FOREIGN KEY: Garantit l'integrite referentielle entre deux tables.
5. CHECK: Verifie qu'une condition est satisfaite pour toutes les lignes.
6. DEFAULT: Definit une valeur par default pour une colonne.

Dans ce laboratoire, nous nous concentrerons principalement sur les contraintes CHECK et sur l'implementation de l'integrite referentielle via des declencheurs.

Type de contrainte	Niveau	Objectif	Exemple
--------------------	--------	----------	---------

Laboratoire 1 - Contraintes et Declencheurs SQL

NOT NULL	Colonne	Eviter les valeurs manquantes	NrSIS NOT NULL
UNIQUE	Colonne/Table	Garantir l'unicite	UNIQUE(NrSIS)
PRIMARY KEY	Table	Identifier uniquement	PRIMARY KEY(NrSIS)
FOREIGN KEY	Table	Integrite referentielle	FOREIGN KEY(NrSIS) REFERENCES
CHECK	Colonne/Table	Valider les donnees	CHECK (Sexe IN ('F', 'M'))
DEFAULT	Colonne	Valeur par default	GrpSanguin DEFAULT

Laboratoire 1 - Contraintes et Declencheurs SQL

1.2. Types de declencheurs

Les declencheurs (triggers) peuvent etre classes selon plusieurs criteres:

1. Moment d'execution:

- BEFORE: S'execute avant l'operation (INSERT, UPDATE, DELETE)
- AFTER: S'execute apres l'operation

2. Niveau d'execution:

- FOR EACH ROW: S'execute pour chaque ligne affectee (declencheur de ligne)
- FOR EACH STATEMENT: S'execute une seule fois par instruction (declencheur d'instruction)

3. Evenement declencheur:

- INSERT: Declenche lors de l'insertion de donnees
- UPDATE: Declenche lors de la mise a jour de donnees
- DELETE: Declenche lors de la suppression de donnees

Dans ce laboratoire, nous utiliserons principalement des declencheurs BEFORE et AFTER au niveau des lignes (FOR EACH ROW).

Type	Moment	Acces	Utilisation courante
BEFORE INSERT	Avant insertion	NEW	Validation, generation de v
AFTER INSERT	Apres insertion	NEW	Audit, mise a jour d'autres
BEFORE UPDATE	Avant mise a jour	OLD, NEW	Validation des modificati
AFTER UPDATE	Apres mise a jour	OLD, NEW	Propagation des modifica
BEFORE DELETE	Avant suppression	OLD	Validation, suppression en c
AFTER DELETE	Apres suppression	OLD	Audit, nettoyage

1.3. Importance dans une base de donnees hospitaliere

Dans un contexte hospitalier, l'integrite et la coherence des donnees sont cruciales pour plusieurs raisons:

1. Securite des patients: Des donnees incorrectes peuvent conduire a des erreurs medicales graves.
2. Conformite reglementaire: Les donnees medicales sont soumises a des reglementations strictes.
3. Efficacite operationnelle: Des donnees coherentes permettent un fonctionnement efficace de l'hopital.
4. Recherche medicale: Des donnees fiables sont essentielles pour la recherche et l'analyse.

Les contraintes et declencheurs jouent un role essentiel pour garantir:

- L'exactitude des informations patient (sexe, groupe sanguin, etc.)

Laboratoire 1 - Contraintes et Declencheurs SQL

- La coherence des references entre patients, medecins et services
- L'automatisation de processus administratifs (attribution de numeros, mise a jour des compteurs)
- La protection contre la perte de donnees lors des operations de suppression

Attention: Les erreurs dans les donnees medicales peuvent avoir des consequences graves sur la sante des patients. Par exemple, une erreur dans le groupe sanguin pourrait entrainer une transfusion incompatible.

Note: Dans ce laboratoire, nous allons implementer plusieurs contraintes et declencheurs pour une base de donnees hospitaliere afin de garantir l'integrite des donnees et d'automatiser certaines operations.

Illustration des contraintes

Table Patients:

- Contrainte sur le sexe: CHECK (Sexe IN ('F', 'M'))
- Contrainte sur l'etat civil: CHECK (EtatCivil IN ('C', 'M', 'D', 'V'))
- Contrainte sur le groupe sanguin: CHECK (GrpSanguin IN ('A', 'B', 'O', 'AB'))
- Contrainte sur le compte bancaire: CHECK (TO_NUMBER(SUBSTR(CptBancaire, 11, 2)) = MOD(...))

2. Contraintes applicatives

Les contraintes applicatives sont des regles qui limitent les valeurs acceptables pour les colonnes d'une table. Dans ce laboratoire, nous avons implemente plusieurs contraintes a la table Patients pour garantir l'integrite des donnees.

Les contraintes applicatives sont essentielles pour:

1. Prevenir les erreurs de saisie
2. Garantir la coherence des donnees
3. Simplifier la logique applicative
4. Ameliorer les performances des requetes

Oracle Database offre plusieurs mecanismes pour implementer ces contraintes, notamment via la clause CHECK qui permet de definir des conditions que les donnees doivent respecter.

Syntaxe generale des contraintes CHECK

```
ALTER TABLE nom_table
ADD CONSTRAINT nom_contrainte
CHECK (condition);

-- Ou lors de la creation de table
CREATE TABLE nom_table (
    colonne1 type_donnees,
    colonne2 type_donnees,
    ...
    CONSTRAINT nom_contrainte CHECK (condition)
);
```

2.1. Contrainte sur le sexe

Cette contrainte garantit que la valeur de la colonne Sexe est soit 'F' (Feminin) soit 'M' (Masculin). Cela permet d'eviter les erreurs de saisie et de maintenir la coherence des donnees.

L'utilisation de l'operateur IN permet de specifier une liste de valeurs autorisees de maniere concise et lisible. Cette approche est preferable a l'utilisation d'operateurs de comparaison multiples (OR, AND) pour des listes de valeurs.

```
ALTER TABLE Patients ADD CONSTRAINT CHK_Patients_Sexe
CHECK (Sexe IN ('F', 'M'));
```

Exemple d'utilisation

Laboratoire 1 - Contraintes et Declencheurs SQL

```
-- Requete valide
INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
VALUES ('123456789012', 'Test', 'Test', 'F');

-- Requete invalide (declenchera une erreur)
INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
VALUES ('123456789012', 'Test', 'Test', 'X');
-- ORA-02290: check constraint (CHK_PATIENTS_SEXE) violated
```

Conseil: Pour les donnees existantes, assurez-vous qu'elles respectent la contrainte avant de l'ajouter. Vous pouvez utiliser une requete comme: `SELECT * FROM Patients WHERE Sexe NOT IN ('F', 'M');`

2.2. Contrainte sur l'etat civil

Cette contrainte garantit que la valeur de la colonne EtatCivil est l'une des suivantes :

- 'C' : Celibataire
- 'M' : Marie(e)
- 'D' : Divorce(e)
- 'V' : Veuf/Veuve

Cela permet de standardiser les valeurs et de faciliter les requetes et les analyses. En limitant les valeurs possibles, on evite les erreurs de saisie et on simplifie les operations de filtrage et de regroupement.

```
ALTER TABLE Patients ADD CONSTRAINT CHK_Patients_EtatCivil
CHECK (EtatCivil IN ('C', 'M', 'D', 'V'));
```

Exemple d'utilisation et impact sur les requetes

```
-- Exemple de requete facilitee par la standardisation
-- Nombre de patients par etat civil
SELECT EtatCivil,
       DECODE(EtatCivil, 'C', 'Celibataire',
                  'M', 'Marie(e)',
                  'D', 'Divorce(e)',
                  'V', 'Veuf/Veuve') AS Description,
       COUNT(*) AS Nombre
FROM Patients
GROUP BY EtatCivil
ORDER BY COUNT(*) DESC;
```

Laboratoire 1 - Contraintes et Declencheurs SQL

2.3. Contrainte sur le groupe sanguin

Cette contrainte garantit que la valeur de la colonne GrpSanguin est l'une des suivantes : 'A', 'B', 'O' ou 'AB'. De plus, une valeur par défaut 'A' est définie pour cette colonne.

Le groupe sanguin est une information médicale critique qui doit être strictement contrôlée pour éviter des erreurs potentiellement fatales lors des transfusions sanguines. Cette contrainte est donc particulièrement importante dans un contexte hospitalier.

```
ALTER TABLE Patients MODIFY (GrpSanguin DEFAULT 'A');  
ALTER TABLE Patients ADD CONSTRAINT CHK_Patients_GroupeSanguin  
CHECK (GrpSanguin IN ('A', 'B', 'O', 'AB'));
```

Explication détaillée

Cette implémentation comporte deux parties distinctes:

1. Définition d'une valeur par défaut:

La clause MODIFY avec DEFAULT 'A' définit 'A' comme valeur par défaut pour la colonne GrpSanguin. Cela signifie que si aucune valeur n'est spécifiée lors de l'insertion d'un nouveau patient, le groupe sanguin sera automatiquement défini comme 'A'.

2. Contrainte de vérification:

La contrainte CHECK limite les valeurs possibles aux quatre groupes sanguins principaux: A, B, O et AB. Cette liste est exhaustive et n'inclut pas les facteurs Rhesus (+ ou -) qui pourraient être gérés dans une colonne séparée pour plus de précision.

Attention: Bien que 'A' soit défini comme valeur par défaut, il est fortement recommandé de toujours spécifier explicitement le groupe sanguin réel du patient pour des raisons de sécurité médicale.

2.4. Contrainte sur le compte bancaire

Cette contrainte vérifie que le compte bancaire est valide en s'assurant que les deux derniers caractères correspondent au reste de la division des dix premiers caractères par 97. C'est une règle de validation couramment utilisée pour les comptes bancaires en Europe, notamment pour les codes IBAN (International Bank Account Number).

Cette validation mathématique permet de détecter immédiatement les erreurs de saisie ou de transmission, avant même que les informations bancaires ne soient utilisées pour des opérations financières.

```
ALTER TABLE Patients ADD CONSTRAINT CHK_Patients_CompteBancaire  
CHECK (TO_NUMBER(SUBSTR(CptBancaire, 11, 2)) = MOD(TO_NUMBER(SUBSTR(CptBancaire, 1, 10)), 97));
```

Explication technique

La contrainte utilise plusieurs fonctions SQL:

Laboratoire 1 - Contraintes et Declencheurs SQL

1. SUBSTR(CptBancaire, 11, 2): Extrait les deux derniers caracteres du compte bancaire (positions 11 et 12).
2. TO_NUMBER(): Convertit la chaine de caracteres extraite en nombre.
3. SUBSTR(CptBancaire, 1, 10): Extrait les dix premiers caracteres du compte bancaire.
4. MOD(nombre, 97): Calcule le reste de la division du nombre par 97.

La contrainte verifie que les deux derniers chiffres correspondent exactement au reste de la division des dix premiers chiffres par 97. Cette methode de validation, appelee "modulo 97", est un algorithme standard pour la verification des numeros de compte bancaire.

Note: Cette contrainte utilise les fonctions SUBSTR pour extraire des parties de la chaine et MOD pour calculer le reste de la division. Elle garantit que le compte bancaire suit un format specifique et contient une somme de controle valide.

Exemple de validation

```
-- Exemple de compte bancaire valide: 1234567890-01
-- Les 10 premiers chiffres: 1234567890
-- Les 2 derniers chiffres: 01
-- Calcul: 1234567890 % 97 = 1
-- Comme 01 = 1, le compte est valide

-- Test avec un compte valide
INSERT INTO Patients (NrSIS, Nom, Prenom, CptBancaire)
VALUES ('123456789012', 'Dupont', 'Jean', '123456789001');

-- Test avec un compte invalide
INSERT INTO Patients (NrSIS, Nom, Prenom, CptBancaire)
VALUES ('123456789012', 'Dupont', 'Jean', '123456789002');
-- ORA-02290: check constraint (CHK_PATIENTS_COMPTEBANCAIRE) violated
```

Laboratoire 1 - Contraintes et Declencheurs SQL

2.5. Contraintes d'integrite referentielle

L'integrite referentielle garantit que les relations entre les tables sont maintenues de maniere coherente. Elle assure que les references d'une table a une autre sont valides.

Dans une base de donnees relationnelle standard, l'integrite referentielle est generalement implementee via des contraintes FOREIGN KEY. Cependant, dans certains cas, notamment lorsqu'on a besoin de logique personnalisee ou de comportements specifiques lors des operations de mise a jour ou de suppression, les declencheurs peuvent offrir une alternative plus flexible.

Aspect	Contrainte FOREIGN KEY	Implementation par declencheur
Declaration	Simple et declarative	Plus complexe, necessite du code PL
Flexibilite	Limitee aux options standard	Entierement personnalisable
Performance	Generalement plus efficace	Peut etre moins efficace
Maintenance	Facile a comprendre	Necessite une documentation clai
Comportements	CASCADE, SET NULL, NO ACTION	Tout comportement programmabl
Validation	Automatique par le SGBD	Doit etre soigneusement testee

Exemple d'implementation avec FOREIGN KEY

```
-- Implementation standard avec FOREIGN KEY
ALTER TABLE PatientsMedecins
ADD CONSTRAINT FK_PatientsMedecins_Patients
FOREIGN KEY (NrSIS) REFERENCES Patients(NrSIS)
ON DELETE CASCADE;

-- Equivalent avec un declencheur (voir section 3.4)
CREATE OR REPLACE TRIGGER PatientsDelNrSIS
BEFORE DELETE ON Patients
FOR EACH ROW
BEGIN
    DELETE FROM PatientsMedecins
    WHERE NrSIS = :OLD.NrSIS;
END;
```

Conseil: Dans notre laboratoire, nous utilisons des declencheurs pour implementer l'integrite referentielle car ils nous permettent d'ajouter des comportements personnalises qui vont au-dela des options standard CASCADE, SET NULL et NO ACTION.

3. Declencheurs (Triggers)

Les declencheurs (triggers) sont des procedures stockees qui s'executent automatiquement en reponse a certains evenements sur une table ou une vue. Ils constituent un mecanisme puissant pour implementer des regles metier complexes et maintenir l'integrite des donnees.

Dans Oracle Database, les declencheurs peuvent etre declenches par differents types d'operations (INSERT, UPDATE, DELETE) et a differents moments (BEFORE, AFTER). Ils peuvent egalement s'executer au niveau de l'instruction (statement-level) ou au niveau de chaque ligne (row-level).

Dans ce laboratoire, nous avons implemente quatre declencheurs pour automatiser certaines operations et maintenir l'integrite referentielle entre les tables de notre base de donnees hospitaliere.

Syntaxe generale des declencheurs

```
CREATE [OR REPLACE] TRIGGER nom_declencheur
{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE [OF colonne] | DELETE}
[OR {INSERT | UPDATE [OF colonne] | DELETE}...]
ON nom_table
[REFERENCING {OLD AS ancien | NEW AS nouveau}]
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
    -- Declaration des variables
BEGIN
    -- Corps du declencheur
EXCEPTION
    -- Gestion des exceptions
END;
```

3.1. Types de declencheurs

Oracle Database prend en charge plusieurs types de declencheurs, chacun ayant des cas d'utilisation specifiques:

Type de declencheur	Description	Cas d'utilisation
BEFORE	Execute avant l'operation DML	Validation des donnees, modification des valeurs avant l'insertion
AFTER	Execute apres l'operation DML	Journalisation, notification, mise a jour des donnees
INSTEAD OF	Remplace l'operation DML sur une vue	Operations DML sur des vues non modifiables

Laboratoire 1 - Contraintes et Declencheurs SQL

Statement-level	Execute une fois par instruction SQL	Operations globales, statistiques, journaux
Row-level (FOR EACH ROW)	Execute pour chaque ligne affectee	Validation ou modification au niveau de la ligne

Note: Dans notre laboratoire, nous utilisons principalement des declencheurs BEFORE et AFTER au niveau des lignes (FOR EACH ROW) pour implementer des regles metier specifiques.

Declencheurs implementes dans notre laboratoire

1. PatientsInsertNrSis: Genere automatiquement un numero SIS lors de l'insertion d'un patient
Type: BEFORE INSERT, Row-level
2. HopitauxServicesNbMed: Met a jour le nombre de medecins dans un service
Type: AFTER INSERT/UPDATE/DELETE, Row-level
3. PatientsUpdateNrSis: Propage les modifications de la date de naissance
Type: AFTER UPDATE, Row-level
4. PatientsDelNrSIS: Supprime en cascade les references lors de la suppression
Type: BEFORE DELETE, Row-level

Laboratoire 1 - Contraintes et Declencheurs SQL

3.2. PatientsInsertNrSis

Ce declencheur s'execute avant l'insertion d'un nouveau patient lorsque le numero SIS n'est pas specifie. Il genere automatiquement un numero SIS base sur la date de naissance du patient et un compteur incrementale.

Le numero SIS est compose de deux parties:

1. Les 6 premiers caracteres representent la date de naissance au format YYMMDD
2. Les 5 derniers caracteres representent un numero sequentiel

Cette approche garantit que chaque patient recoit un numero SIS unique tout en incorporant des informations utiles (date de naissance) dans ce numero.

```
CREATE OR REPLACE TRIGGER PatientsInsertNrSis
BEFORE INSERT ON Patients
FOR EACH ROW
WHEN (NEW.NrSIS IS NULL)
DECLARE
    v_last_nr NUMBER;
    v_date_part VARCHAR2(6);
    v_nr_part VARCHAR2(5);
BEGIN
    -- Extraction de la date de naissance au format YYMMDD
    v_date_part := TO_CHAR(:NEW.DateNaissance, 'YYMMDD');

    -- Recuperation du dernier numero utilise
    SELECT NVL(MAX(TO_NUMBER(SUBSTR(NrSIS, 7, 5))), 0)
    INTO v_last_nr
    FROM Patients
    WHERE SUBSTR(NrSIS, 1, 6) = v_date_part;

    -- Incrementation du numero
    v_last_nr := v_last_nr + 1;

    -- Formatage du numero sur 5 chiffres
    v_nr_part := LPAD(v_last_nr, 5, '0');

    -- Assignment du NrSIS
    :NEW.NrSIS := v_date_part || v_nr_part;
END;
```

Explication detaillee

Ce declencheur utilise plusieurs concepts PL/SQL importants:

1. Clause WHEN: La condition WHEN (NEW.NrSIS IS NULL) limite l'execution du declencheur aux cas ou le numero SIS n'est pas specifie, permettant ainsi des insertions manuelles si necessaire.

Laboratoire 1 - Contraintes et Declencheurs SQL

2. Variables de liaison :NEW et :OLD: Ces variables speciales permettent d'accéder aux valeurs nouvelles (:NEW) et anciennes (:OLD) des colonnes de la ligne en cours de traitement.

3. Fonctions SQL integrees:

- TO_CHAR(): Convertit la date en chaine de caracteres selon un format specifie
- SUBSTR(): Extrait une sous-chaine a partir d'une position donnee
- LPAD(): Remplit une chaine a gauche avec un caractere specifie jusqu'a atteindre une longueur donnee
- NVL(): Remplace NULL par une valeur par default

4. Requete SQL imbriquee: Utilise pour trouver le dernier numero sequentiel utilise pour une date de naissance donnee.

Exemple d'utilisation

```
-- Insertion d'un patient sans specifier le NrSIS
INSERT INTO Patients (Nom, Prenom, DateNaissance, Sexe, EtatCivil, GrpSanguin)
VALUES ('Dupont', 'Jean', TO_DATE('1975-12-30', 'YYYY-MM-DD'), 'M', 'M', 'A');

-- Le declencheur genere automatiquement le NrSIS: 751230-00001
-- (si c'est le premier patient ne le 30/12/1975)

-- Verification
SELECT NrSIS, Nom, Prenom, DateNaissance
FROM Patients
WHERE Nom = 'Dupont' AND Prenom = 'Jean';
```

Conseil: Ce declencheur simplifie le processus d'insertion en automatisant la generation du numero SIS, tout en garantissant son unicite. Il reduit egalement le risque d'erreurs humaines lors de la saisie manuelle des numeros SIS.

Laboratoire 1 - Contraintes et Declencheurs SQL

3.3. HopitauxServicesNbreMed

Ce declencheur met a jour automatiquement le nombre de medecins (NbreMedecins) dans la table HopitauxServices apres chaque insertion, mise a jour ou suppression dans la table MedecinsServices.

Il s'agit d'un declencheur AFTER qui s'execute apres que les modifications ont ete apportees a la table MedecinsServices. Ce type de declencheur est ideal pour maintenir des valeurs derivees ou agregees, comme le nombre de medecins dans un service.

```
CREATE OR REPLACE TRIGGER HopitauxServicesNbreMed
AFTER INSERT OR UPDATE OR DELETE ON MedecinsServices
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    IF INSERTING OR UPDATING THEN
        -- Compter le nombre de medecins pour ce service dans cet hopital
        SELECT COUNT(*) INTO v_count FROM MedecinsServices WHERE NrHopital = :NEW.NrHopital AND
NrService = :NEW.NrService;

        -- Mettre a jour le nombre de medecins
        UPDATE HopitauxServices SET NbreMedecins = v_count WHERE NrHopital = :NEW.NrHopital AND
NrService = :NEW.NrService;
    ELSIF DELETING THEN
        -- Compter le nombre de medecins pour ce service dans cet hopital
        SELECT COUNT(*) INTO v_count FROM MedecinsServices WHERE NrHopital = :OLD.NrHopital AND
NrService = :OLD.NrService;

        -- Mettre a jour le nombre de medecins
        UPDATE HopitauxServices SET NbreMedecins = v_count WHERE NrHopital = :OLD.NrHopital AND
NrService = :OLD.NrService;
    END IF;
END;
```

Explication detaillee

Ce declencheur utilise plusieurs concepts avances:

1. Declencheur multi-evenements: Il gere trois types d'operations (INSERT, UPDATE, DELETE) dans un seul declencheur grace a la clause OR.
2. Structure conditionnelle: Utilise IF-ELSIF-END IF pour executer un code different selon le type d'operation declenchante.
3. Predicats implicites: Les predicats INSERTING, UPDATING et DELETING permettent de determiner quel type d'operation a declenche le trigger.
4. Requetes de comptage: Utilise COUNT(*) pour calculer le nombre total de medecins dans un service

Laboratoire 1 - Contraintes et Declencheurs SQL

specifique.

5. Mise a jour de tables liees: Effectue une mise a jour dans une table differente de celle qui a declenche l'evenement.

Avantages et inconvenients

Avantages	Inconvenients
Maintient automatiquement les valeurs agregees	Peut impacter les performances lors d'operations massives
Garantit la coherence des donnees	Complexite accrue de la logique de base de donnees
Centralise la logique de mise a jour	Risque de declencheurs en cascade si mal concu
Reduit le code applicatif necessaire	Peut etre difficile a deboguer

Attention: Attention aux declencheurs en cascade: si une mise a jour dans HopitauxServices declenchait un autre trigger qui modifierait MedecinsServices, cela pourrait creer une boucle infinie. Oracle limite a 32 le nombre de declencheurs en cascade pour eviter ce probleme.

Laboratoire 1 - Contraintes et Declencheurs SQL

3.4. PatientsUpdateNrSis

Ce declencheur s'execute apres la mise a jour de la date de naissance d'un patient. Il met a jour le numero SIS en fonction de la nouvelle date de naissance et propage cette modification a toutes les tables qui referencent ce numero.

Ce declencheur simule le comportement de ON UPDATE CASCADE qui n'est pas directement supporte par Oracle Database. Il permet de maintenir l'integrite referentielle lors des modifications de cles primaires.

```
CREATE OR REPLACE TRIGGER PatientsUpdateNrSis
AFTER UPDATE OF DateNaissance ON Patients
FOR EACH ROW
DECLARE
    v_old_nrsis VARCHAR2(11);
    v_new_nrsis VARCHAR2(11);
    v_date_part VARCHAR2(6);
BEGIN
    -- Sauvegarde de l'ancien NrSIS
    v_old_nrsis := :OLD.NrSIS;

    -- Extraction de la nouvelle date de naissance au format YYMMDD
    v_date_part := TO_CHAR(:NEW.DateNaissance, 'YYMMDD');

    -- Construction du nouveau NrSIS
    v_new_nrsis := v_date_part || SUBSTR(:OLD.NrSIS, 7, 5);

    -- Mise a jour du NrSIS dans la table Patients
    UPDATE Patients SET NrSIS = v_new_nrsis WHERE NrSIS = v_old_nrsis;

    -- Mise a jour des references dans les autres tables
    UPDATE PatientsMedecins SET NrSIS = v_new_nrsis WHERE NrSIS = v_old_nrsis;

    -- Mise a jour des references dans la table Allergies
    UPDATE Allergies SET NrSIS = v_new_nrsis WHERE NrSIS = v_old_nrsis;
END;
```

Explication technique

Ce declencheur presente plusieurs aspects techniques interessants:

1. Declencheur specifique a une colonne: La clause UPDATE OF DateNaissance limite l'execution du declencheur aux mises a jour de la colonne DateNaissance uniquement.
2. Propagation des modifications: Le declencheur propage les modifications de la cle primaire (NrSIS) a toutes les tables qui la referencent, simulant ainsi le comportement ON UPDATE CASCADE.
3. Mutating table: Ce declencheur peut potentiellement rencontrer l'erreur "mutating table" car il tente de modifier la table Patients qui est deja en cours de modification. Dans certains cas, cette approche peut

Laboratoire 1 - Contraintes et Declencheurs SQL

nécessiter une refactorisation pour utiliser des déclencheurs composés (au niveau de l'instruction et au niveau des lignes) ou des tables temporaires.

Note: Ce déclencheur simule le comportement de ON UPDATE CASCADE qui n'est pas directement supporté par Oracle. Il propage les modifications du numéro SIS à toutes les tables qui référencent ce numéro.

Diagramme de sequence d'execution

Sequence d'execution du déclencheur

1. Mise à jour de la date de naissance d'un patient
2. Execution du déclencheur AFTER UPDATE
3. Calcul du nouveau numéro SIS basé sur la nouvelle date
4. Mise à jour du numéro SIS dans la table Patients
5. Propagation de la modification aux tables PatientsMedecins et Allergies

Laboratoire 1 - Contraintes et Declencheurs SQL

3.5. PatientsDelNrSIS

Ce declencheur s'execute avant la suppression d'un patient. Il supprime automatiquement toutes les references a ce patient dans les autres tables, simulant ainsi le comportement de ON DELETE CASCADE.

Ce type de declencheur est particulierement utile pour maintenir l'integrite referentielle dans les bases de donnees Oracle, ou les contraintes de cle etrangere avec CASCADE ne sont pas toujours suffisantes pour des logiques metier complexes.

```
CREATE OR REPLACE TRIGGER PatientsDelNrSIS
BEFORE DELETE ON Patients
FOR EACH ROW
BEGIN
    -- Suppression des enregistrements lies dans les autres tables
    DELETE FROM PatientsMedecins WHERE NrSIS = :OLD.NrSIS;

    -- Suppression des enregistrements lies dans la table Allergies
    DELETE FROM Allergies WHERE NrSIS = :OLD.NrSIS;
END;
```

Comparaison avec les contraintes de cle etrangere

Aspect	Declencheur BEFORE DELETE	Contrainte FOREIGN KEY avec ON DELETE
Flexibilite	Permet une logique personnalisee	Comportement fixe defini par le SGBD
Journalisation	Peut inclure une journalisation personnalisee	Pas de journalisation integree
Validation	Peut inclure des validations supplementaires	Validation limitee aux contraintes existantes
Performance	Potentiellement moins performant	Optimise par le moteur de base de donnees
Maintenance	Necessite une documentation claire	Auto-documentee dans le schéma
Traitement conditionnel	Peut appliquer une logique conditionnelle	Toujours applique de la meme maniere

Exemple d'utilisation avancee

```
-- Version amelioree avec journalisation et validation
CREATE OR REPLACE TRIGGER PatientsDelNrSIS
BEFORE DELETE ON Patients
FOR EACH ROW
DECLARE
    v_count_med NUMBER;
    v_count_all NUMBER;
BEGIN
    -- Verification du nombre d'enregistrements a supprimer
    SELECT COUNT(*) INTO v_count_med FROM PatientsMedecins WHERE NrSIS = :OLD.NrSIS;
    SELECT COUNT(*) INTO v_count_all FROM Allergies WHERE NrSIS = :OLD.NrSIS;
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
-- Journalisation des suppressions
INSERT INTO LogSuppressions (DateSuppression, TableSource, CleSource, NombreReferencesSupp)
VALUES (SYSDATE, 'Patients', :OLD.NrSIS, v_count_med + v_count_all);

-- Suppression des enregistrements lies
DELETE FROM PatientsMedecins WHERE NrSIS = :OLD.NrSIS;
DELETE FROM Allergies WHERE NrSIS = :OLD.NrSIS;

-- On pourrait ajouter une validation ou une notification ici
END;
```

Conseil: Pour les operations critiques comme la suppression de donnees patient, il est recommande d'ajouter une journalisation detaillee afin de pouvoir auditer ces operations et eventuellement les annuler si necessaire.

Conseil: Pour les operations critiques comme la suppression de donnees patient, il est recommande d'ajouter une journalisation detaillee afin de pouvoir auditer ces operations et eventuellement les annuler si necessaire.

4. Tests et validation

Pour garantir le bon fonctionnement des contraintes et des declencheurs implementes, il est essentiel de mettre en place une strategie de tests rigoureuse. Cette section presente les differentes approches de test et les scenarios a verifier.

4.1. Strategie de test

Une strategie de test efficace pour les contraintes et declencheurs doit inclure:

1. Tests unitaires: Verifier chaque contrainte et declencheur individuellement
2. Tests d'integration: Verifier les interactions entre les differentes contraintes et declencheurs
3. Tests de charge: Evaluer les performances lors d'operations massives
4. Tests de regression: S'assurer que les modifications n'affectent pas les fonctionnalites existantes

Les tests doivent couvrir a la fois les cas de succes (donnees valides) et les cas d'echec (donnees invalides).

Methodologie de test

1. Preparer l'environnement de test (jeu de donnees initial)
2. Executer les operations a tester
3. Verifier les resultats (valeurs dans les tables, messages d'erreur)
4. Nettoyer l'environnement (rollback ou suppression des donnees de test)
5. Documenter les resultats des tests

4.2. Tests des contraintes

Pour tester les contraintes applicatives, nous devons verifier qu'elles empechent correctement l'insertion de donnees invalides et permettent l'insertion de donnees valides. Voici les scenarios de test pour chaque contrainte:

Contrainte	Scenario de test	Resultat attendu
CHK_Patients_Sexe	Insertion avec Sexe = 'F'	Succes
CHK_Patients_Sexe	Insertion avec Sexe = 'M'	Succes
CHK_Patients_Sexe	Insertion avec Sexe = 'X'	Echec (ORA-02290)
CHK_Patients_EtatCivil	Insertion avec EtatCivil = 'C'	Succes
CHK_Patients_EtatCivil	Insertion avec EtatCivil = 'Z'	Echec (ORA-02290)
CHK_Patients_GroupeSanguin	Insertion avec GrpSanguin = 'AB'	Succes

Laboratoire 1 - Contraintes et Declencheurs SQL

CHK_Patients_GroupeSanguin	Insertion avec GrpSanguin = 'XY'	Echec (ORA-02290)
CHK_Patients_CompteBancaire	Insertion avec CptBancaire valide	Succes
CHK_Patients_CompteBancaire	Insertion avec CptBancaire invalide	Echec (ORA-02290)

Script de test des contraintes

```
-- Preparation: Activer l'affichage des erreurs
SET SERVEROUTPUT ON;

-- Fonction utilitaire pour tester les contraintes
CREATE OR REPLACE PROCEDURE test_constraint(
    p_test_name VARCHAR2,
    p_sql VARCHAR2
) AS
    v_success BOOLEAN := TRUE;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Test: ' || p_test_name);
    BEGIN
        EXECUTE IMMEDIATE p_sql;
        DBMS_OUTPUT.PUT_LINE('  Resultat: SUCCES (operation reussie)');
    EXCEPTION
        WHEN OTHERS THEN
            v_success := FALSE;
            DBMS_OUTPUT.PUT_LINE('  Resultat: ECHEC - ' || SQLERRM);
    END;

    -- Annuler les modifications si le test a reussi
    IF v_success THEN
        ROLLBACK;
    END IF;
END;
/

-- Tests de la contrainte sur le sexe
BEGIN
    -- Test valide avec 'F'
    test_constraint(
        'CHK_Patients_Sexe - Valeur valide F',
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
        VALUES (''999999999901'', ''Test'', ''Test'', ''F'')'
    );

    -- Test valide avec 'M'
    test_constraint(
        'CHK_Patients_Sexe - Valeur valide M',
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
        VALUES (''999999999902'', ''Test'', ''Test'', ''M'')'
    );
END;
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
-- Test invalide avec 'X'
test_constraint(
    'CHK_Patients_Sexe - Valeur invalide X',
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
    VALUES (''9999999999903'', ''Test'', ''Test'', ''X'')'
);
END;
/
```

Laboratoire 1 - Contraintes et Declencheurs SQL

4.3. Tests des declencheurs

Pour tester les declencheurs, nous devons verifier qu'ils s'executent correctement et produisent les resultats attendus. Voici les scenarios de test pour chaque declencheur:

Declencheur	Scenario de test	Verification
PatientsInsertNrSis	Insertion sans specifier NrSIS	Verifier que NrSIS est genere automatiquement
PatientsInsertNrSis	Insertion en specifiant NrSIS	Verifier que NrSIS specifie est conserve
HopitauxServicesNbreMed	Ajout d'un medecin a un service	Verifier que NbreMedecins est incremente
HopitauxServicesNbreMed	Suppression d'un medecin d'un service	Verifier que NbreMedecins est decremente
PatientsUpdateNrSis	Modification de la date de naissance	Verifier que NrSIS est mis a jour dans toutes les tables
PatientsDelNrSIS	Suppression d'un patient	Verifier que les references sont supprimees

Script de test des declencheurs

```
-- Test du declencheur PatientsInsertNrSis
DECLARE
    v_nrsis Patients.NrSIS%TYPE;
    v_date_part VARCHAR2(6);
BEGIN
    -- Insertion sans specifier NrSIS
    INSERT INTO Patients (Nom, Prenom, DateNaissance, Sexe, EtatCivil, GrpSanguin)
    VALUES ('Test', 'Trigger', TO_DATE('1980-05-15', 'YYYY-MM-DD'), 'M', 'C', 'O')
    RETURNING NrSIS INTO v_nrsis;

    -- Verification du format du NrSIS genere
    v_date_part := TO_CHAR(TO_DATE('1980-05-15', 'YYYY-MM-DD'), 'YYMMDD');

    DBMS_OUTPUT.PUT_LINE('NrSIS genere: ' || v_nrsis);
    DBMS_OUTPUT.PUT_LINE('Date part attendue: ' || v_date_part);

    IF SUBSTR(v_nrsis, 1, 6) = v_date_part THEN
        DBMS_OUTPUT.PUT_LINE('Test PatientsInsertNrSis: SUCCES');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Test PatientsInsertNrSis: ECHEC');
    END IF;

    -- Nettoyage
    DELETE FROM Patients WHERE NrSIS = v_nrsis;
    COMMIT;
END;
/

-- Test du declencheur HopitauxServicesNbreMed
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
DECLARE
    v_count_before NUMBER;
    v_count_after  NUMBER;
    v_test_hopital NUMBER := 1;
    v_test_service  NUMBER := 1;
    v_test_medecin  NUMBER := 999;
BEGIN
    -- Nombre de medecins avant
    SELECT NbreMedecins INTO v_count_before
    FROM HopitauxServices
    WHERE NrHopital = v_test_hopital AND NrService = v_test_service;

    -- Ajout d'un medecin
    INSERT INTO MedecinsServices (NrMedecin, NrHopital, NrService)
    VALUES (v_test_medecin, v_test_hopital, v_test_service);

    -- Nombre de medecins apres
    SELECT NbreMedecins INTO v_count_after
    FROM HopitauxServices
    WHERE NrHopital = v_test_hopital AND NrService = v_test_service;

    -- Verification
    DBMS_OUTPUT.PUT_LINE('Nombre avant: ' || v_count_before);
    DBMS_OUTPUT.PUT_LINE('Nombre apres: ' || v_count_after);

    IF v_count_after = v_count_before + 1 THEN
        DBMS_OUTPUT.PUT_LINE('Test HopitauxServicesNbreMed (INSERT): SUCCES');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Test HopitauxServicesNbreMed (INSERT): ECHEC');
    END IF;

    -- Nettoyage
    DELETE FROM MedecinsServices
    WHERE NrMedecin = v_test_medecin
    AND NrHopital = v_test_hopital
    AND NrService = v_test_service;

    COMMIT;
END;
/
```

Bonnes pratiques de test

Lors des tests de contraintes et de declencheurs, il est important de suivre ces bonnes pratiques:

1. Isoler les tests: Chaque test doit etre independant des autres
2. Utiliser des transactions: Encadrer les tests par BEGIN/ROLLBACK pour eviter de polluer la base de donnees
3. Automatiser les tests: Creer des scripts de test reutilisables
4. Tester les cas limites: Ne pas se limiter aux cas nominaux

Laboratoire 1 - Contraintes et Declencheurs SQL

5. Verifier les effets de bord: S'assurer que les declencheurs n'ont pas d'effets indesirables

Attention: Les tests doivent etre executes dans un environnement de test dedie, jamais directement en production. Assurez-vous de disposer d'une sauvegarde recente avant d'executer des tests sur une base de donnees existante.

5. Conclusion

Dans ce laboratoire, nous avons explore en profondeur les mecanismes de contraintes et de declencheurs dans Oracle Database, en les appliquant a un contexte hospitalier. Ces outils puissants permettent de garantir l'integrite des donnees et d'automatiser des operations complexes.

Les contraintes applicatives nous ont permis de definir des regles metier strictes pour les donnees patient, comme les valeurs acceptables pour le sexe, l'etat civil et le groupe sanguin, ainsi que la validation des comptes bancaires. Ces contraintes constituent une premiere ligne de defense contre les erreurs de saisie et les incoherences.

Les declencheurs nous ont offert une flexibilite supplementaire pour implementer des comportements personnalises, comme la generation automatique de numeros SIS, la mise a jour des compteurs, et la propagation des modifications ou suppressions a travers plusieurs tables. Ils permettent d'encapsuler la logique metier directement dans la base de donnees, garantissant son application systematique.

L'utilisation combinee de ces mecanismes est particulierement pertinente dans un contexte hospitalier, ou l'integrite et la coherence des donnees sont cruciales. Une erreur dans les donnees medicales peut avoir des consequences graves, d'ou l'importance de mettre en place des controles rigoureux.

Enfin, les tests systematiques de ces mecanismes sont essentiels pour s'assurer de leur bon fonctionnement et de leur robustesse face a differents scenarios d'utilisation.

5.1. Resume des objets crees

Objets implements

Contraintes:

- CHK_PATIENTS_SEXE: Limite les valeurs de Sexe a 'F' et 'M'
- CHK_PATIENTS_ETATCIVIL: Limite les valeurs d'EtatCivil a 'C', 'M', 'D' et 'V'
- CHK_PATIENTS_GROUPESANGUIN: Limite les valeurs de GrpSanguin a 'A', 'B', 'O' et 'AB'
- CHK_PATIENTS_COMPTEBANCAIRE: Verifie la validite du compte bancaire

Declencheurs:

- PATIENTSINSERTNSRIS: Genere automatiquement un numero SIS lors de l'insertion
- HOPITAUXSERVICESNBREMED: Met a jour le nombre de medecins dans un service
- PATIENTSUPDATENRSIS: Propage les modifications de la date de naissance
- PATIENTSDELNRSIS: Supprime en cascade les references lors de la suppression

5.2. Perspectives d'amelioration

Laboratoire 1 - Contraintes et Declencheurs SQL

Plusieurs ameliorations pourraient etre apportees a notre implementation:

1. Gestion des erreurs: Ajouter des blocs EXCEPTION dans les declencheurs pour traiter les cas d'erreur de maniere plus elegante.
2. Journalisation: Implementer un systeme de journalisation complet pour suivre toutes les operations critiques.
3. Optimisation des performances: Revoir les declencheurs pour minimiser leur impact sur les performances, notamment lors d'operations massives.
4. Securite renforcee: Ajouter des contraintes supplementaires pour proteger les donnees sensibles.
5. Tests automatises: Developper une suite de tests automatises pour verifier regulierement l'integrite du systeme.

5.3. References

Voici quelques ressources utiles pour approfondir les sujets abordes dans ce laboratoire:

- Oracle Database SQL Language Reference: Documentation officielle sur le langage SQL d'Oracle
- Oracle Database PL/SQL Language Reference: Documentation sur PL/SQL, le langage procedural d'Oracle
- Oracle Database Concepts: Concepts fondamentaux des bases de donnees Oracle
- SQL Performance Tuning: Techniques d'optimisation des performances SQL
- Database Design and Relational Theory: Principes theoriques de la conception de bases de donnees

Annexe: Scripts de test complets

Cette annexe contient des scripts de test complets pour valider le fonctionnement des contraintes et des declencheurs dans le contexte de notre base de donnees hospitaliere. Ces scripts peuvent etre executes dans un environnement de test pour verifier l'integrite du systeme.

A.1. Preparation de l'environnement de test

Avant d'executer les tests, il est necessaire de preparer l'environnement de test en creant des donnees de test. Le script suivant cree un jeu de donnees minimal pour tester les contraintes et les declencheurs.

```
-- Activation de l'affichage des erreurs
SET SERVEROUTPUT ON SIZE 1000000;

-- Creation d'une procedure pour executer les tests
CREATE OR REPLACE PACKAGE test_utils AS
    -- Procedure pour afficher un titre de test
    PROCEDURE print_test_header(p_test_name VARCHAR2);

    -- Procedure pour afficher un resultat de test
    PROCEDURE print_test_result(p_test_name VARCHAR2, p_success BOOLEAN, p_message VARCHAR2
DEFAULT NULL);

    -- Fonction pour executer une requete SQL et capturer les erreurs
    FUNCTION exec_sql(p_sql VARCHAR2) RETURN BOOLEAN;
END test_utils;
/

CREATE OR REPLACE PACKAGE BODY test_utils AS
    PROCEDURE print_test_header(p_test_name VARCHAR2) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('TEST: ' || p_test_name);
        DBMS_OUTPUT.PUT_LINE('=====');
    END print_test_header;

    PROCEDURE print_test_result(p_test_name VARCHAR2, p_success BOOLEAN, p_message VARCHAR2
DEFAULT NULL) IS
    BEGIN
        IF p_success THEN
            DBMS_OUTPUT.PUT_LINE('RESULTAT: SUCCES - ' || p_test_name);
        ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTAT: ECHEC - ' || p_test_name);
        END IF;

        IF p_message IS NOT NULL THEN
            DBMS_OUTPUT.PUT_LINE('MESSAGE: ' || p_message);
        END IF;
    END print_test_result;

    FUNCTION exec_sql(p_sql VARCHAR2) RETURN BOOLEAN IS
    BEGIN
        EXECUTE IMMEDIATE p_sql;
        RETURN TRUE;
    END exec_sql;
END test_utils;
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
        DBMS_OUTPUT.PUT_LINE('-----');
END print_test_result;

FUNCTION exec_sql(p_sql VARCHAR2) RETURN BOOLEAN IS
    v_success BOOLEAN := TRUE;
BEGIN
    EXECUTE IMMEDIATE p_sql;
    RETURN TRUE;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERREUR: ' || SQLERRM);
        RETURN FALSE;
END exec_sql;
END test_utils;
/

-- Preparation des donnees de test
BEGIN
    -- Nettoyage des tables de test (si necessaire)
    BEGIN EXECUTE IMMEDIATE 'DELETE FROM PatientsMedecins WHERE NrSIS LIKE ''TEST%'''; EXCEPTION
WHEN OTHERS THEN NULL; END;
    BEGIN EXECUTE IMMEDIATE 'DELETE FROM Allergies WHERE NrSIS LIKE ''TEST%'''; EXCEPTION WHEN
OTHERS THEN NULL; END;
    BEGIN EXECUTE IMMEDIATE 'DELETE FROM Patients WHERE NrSIS LIKE ''TEST%'''; EXCEPTION WHEN
OTHERS THEN NULL; END;
    BEGIN EXECUTE IMMEDIATE 'DELETE FROM MedecinsServices WHERE NrMedecin LIKE ''TEST%''';
EXCEPTION WHEN OTHERS THEN NULL; END;
    BEGIN EXECUTE IMMEDIATE 'DELETE FROM HopitauxServices WHERE ChefService LIKE ''TEST%''';
EXCEPTION WHEN OTHERS THEN NULL; END;
    BEGIN EXECUTE IMMEDIATE 'DELETE FROM Medecins WHERE NrMedecin LIKE ''TEST%'''; EXCEPTION WHEN
OTHERS THEN NULL; END;

    DBMS_OUTPUT.PUT_LINE('Environnement de test prepare');
END;
/
```

A.2. Tests des contraintes

A.2.1. Test de la contrainte sur le sexe

```
-- Test de la contrainte CHK_Patients_Sexe
DECLARE
    v_success BOOLEAN;
BEGIN
    test_utils.print_test_header('Contrainte CHK_Patients_Sexe');

    -- Test avec valeur valide 'F'
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
        VALUES (''TEST0000001'', ''Test'', ''Test'', ''F'')'
    );
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
test_utils.print_test_result('Insertion avec Sexe = F', v_success);

-- Test avec valeur valide 'M'
v_success := test_utils.exec_sql(
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
    VALUES (''TEST0000002'', ''Test'', ''Test'', ''M'')'
);
test_utils.print_test_result('Insertion avec Sexe = M', v_success);

-- Test avec valeur invalide 'X'
v_success := test_utils.exec_sql(
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
    VALUES (''TEST0000003'', ''Test'', ''Test'', ''X'')'
);
test_utils.print_test_result('Insertion avec Sexe = X (doit echouer)', NOT v_success);

-- Nettoyage
EXECUTE IMMEDIATE 'DELETE FROM Patients WHERE NrSIS LIKE ''TEST%''';
END;
/
```

A.2.2. Test de la contrainte sur l'etat civil

```
-- Test de la contrainte CHK_Patients_EtatCivil
DECLARE
    v_success BOOLEAN;
BEGIN
    test_utils.print_test_header('Contrainte CHK_Patients_EtatCivil');

    -- Test avec valeur valide 'C' (Celibataire)
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, EtatCivil)
        VALUES (''TEST0000001'', ''Test'', ''Test'', ''F'', ''C'')'
    );
    test_utils.print_test_result('Insertion avec EtatCivil = C', v_success);

    -- Test avec valeur valide 'M' (Marie)
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, EtatCivil)
        VALUES (''TEST0000002'', ''Test'', ''Test'', ''M'', ''M'')'
    );
    test_utils.print_test_result('Insertion avec EtatCivil = M', v_success);

    -- Test avec valeur valide 'D' (Divorce)
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, EtatCivil)
        VALUES (''TEST0000003'', ''Test'', ''Test'', ''F'', ''D'')'
    );
    test_utils.print_test_result('Insertion avec EtatCivil = D', v_success);

    -- Test avec valeur valide 'V' (Veuf/Veuve)
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
v_success := test_utils.exec_sql(
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, EtatCivil)
    VALUES (''TEST0000004'', ''Test'', ''Test'', ''M'', ''V'')'
);
test_utils.print_test_result('Insertion avec EtatCivil = V', v_success);

-- Test avec valeur invalide 'X'
v_success := test_utils.exec_sql(
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, EtatCivil)
    VALUES (''TEST0000005'', ''Test'', ''Test'', ''F'', ''X'')'
);
test_utils.print_test_result('Insertion avec EtatCivil = X (doit echouer)', NOT v_success);

-- Nettoyage
EXECUTE IMMEDIATE 'DELETE FROM Patients WHERE NrSIS LIKE ''TEST%''';
END;
/
```

A.2.3. Test de la contrainte sur le groupe sanguin

```
-- Test de la contrainte CHK_Patients_GroupeSanguin
DECLARE
    v_success BOOLEAN;
    v_grp_sanguin Patients.GrpSanguin%TYPE;
BEGIN
    test_utils.print_test_header('Contrainte CHK_Patients_GroupeSanguin');

    -- Test avec valeur valide 'A'
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, GrpSanguin)
        VALUES (''TEST0000001'', ''Test'', ''Test'', ''F'', ''A'')'
    );
    test_utils.print_test_result('Insertion avec GrpSanguin = A', v_success);

    -- Test avec valeur valide 'B'
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, GrpSanguin)
        VALUES (''TEST0000002'', ''Test'', ''Test'', ''M'', ''B'')'
    );
    test_utils.print_test_result('Insertion avec GrpSanguin = B', v_success);

    -- Test avec valeur valide 'O'
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, GrpSanguin)
        VALUES (''TEST0000003'', ''Test'', ''Test'', ''F'', ''O'')'
    );
    test_utils.print_test_result('Insertion avec GrpSanguin = O', v_success);

    -- Test avec valeur valide 'AB'
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, GrpSanguin)
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
VALUES ('TEST0000004', 'Test', 'Test', 'M', 'AB'))'
);
test_utils.print_test_result('Insertion avec GrpSanguin = AB', v_success);

-- Test avec valeur invalide 'XY'
v_success := test_utils.exec_sql(
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, GrpSanguin)
    VALUES ('TEST0000005', 'Test', 'Test', 'F', 'XY'))'
);
test_utils.print_test_result('Insertion avec GrpSanguin = XY (doit echouer)', NOT v_success);

-- Test de la valeur par default
v_success := test_utils.exec_sql(
    'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe)
    VALUES ('TEST0000006', 'Test', 'Test', 'M'))'
);

IF v_success THEN
    -- Verification de la valeur par default
    EXECUTE IMMEDIATE
        'SELECT GrpSanguin FROM Patients WHERE NrSIS = 'TEST0000006''
    INTO v_grp_sanguin;

    IF v_grp_sanguin = 'A' THEN
        test_utils.print_test_result('Valeur par default GrpSanguin = A', TRUE);
    ELSE
        test_utils.print_test_result('Valeur par default GrpSanguin = A', FALSE,
            'Valeur obtenue: ' || v_grp_sanguin);
    END IF;
ELSE
    test_utils.print_test_result('Insertion sans specifier GrpSanguin', FALSE);
END IF;

-- Nettoyage
EXECUTE IMMEDIATE 'DELETE FROM Patients WHERE NrSIS LIKE 'TEST%'';
END;
/
```

A.2.4. Test de la contrainte sur le compte bancaire

```
-- Test de la contrainte CHK_Patients_CompteBancaire
DECLARE
    v_success BOOLEAN;

-- Fonction pour generer un compte bancaire valide
FUNCTION generate_valid_account(p_base NUMBER) RETURN VARCHAR2 IS
    v_base VARCHAR2(10);
    v_check NUMBER;
    v_result VARCHAR2(12);
BEGIN
    -- Formatage de la base sur 10 chiffres
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
v_base := LPAD(p_base, 10, '0');

-- Calcul du reste de la division par 97
v_check := MOD(TO_NUMBER(v_base), 97);

-- Construction du compte complet
v_result := v_base || LPAD(v_check, 2, '0');

RETURN v_result;
END;

-- Fonction pour generer un compte bancaire invalide
FUNCTION generate_invalid_account(p_base NUMBER) RETURN VARCHAR2 IS
    v_base VARCHAR2(10);
    v_check NUMBER;
    v_result VARCHAR2(12);
BEGIN
    -- Formatage de la base sur 10 chiffres
    v_base := LPAD(p_base, 10, '0');

    -- Calcul du reste de la division par 97
    v_check := MOD(TO_NUMBER(v_base), 97);

    -- Utilisation d'un check invalide (check + 1)
    IF v_check = 96 THEN
        v_check := 0;
    ELSE
        v_check := v_check + 1;
    END IF;

    -- Construction du compte invalide
    v_result := v_base || LPAD(v_check, 2, '0');

    RETURN v_result;
END;

BEGIN
    test_utils.print_test_header('Contrainte CHK_Patients_CompteBancaire');

    -- Test avec compte bancaire valide
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, CptBancaire)
        VALUES (''TEST0000001'', ''Test'', ''Test'', ''F'', '' ' ||
generate_valid_account(1234567890) || ''')'
    );
    test_utils.print_test_result('Insertion avec compte bancaire valide', v_success);

    -- Test avec compte bancaire invalide
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, CptBancaire)
        VALUES (''TEST0000002'', ''Test'', ''Test'', ''M'', '' ' ||
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
generate_invalid_account(1234567890) || ''')'
);
test_utils.print_test_result('Insertion avec compte bancaire invalide (doit echouer)', NOT
v_success);

-- Nettoyage
EXECUTE IMMEDIATE 'DELETE FROM Patients WHERE NrSIS LIKE 'TEST%'';
END;
/
```

A.3. Tests des déclencheurs

A.3.1. Test du déclencheur de vérification des allergies

```
-- Test du déclencheur TRG_Verif_Allergies
DECLARE
    v_success BOOLEAN;
BEGIN
    test_utils.print_test_header('Déclencheur TRG_Verif_Allergies');

    -- Préparation des données de test
    -- Insertion d'un pays
    v_success := test_utils.exec_sql(
        'INSERT INTO Pays (CodePays, Libelle) VALUES ('TE', 'Test')'
    );

    -- Insertion d'une adresse
    v_success := test_utils.exec_sql(
        'INSERT INTO Adresses (CodeAdresse, Adresse, Localite, CodePostal)
        VALUES (9999, 'Rue de Test', 'TestVille', '12345')'
    );

    -- Insertion d'une mutuelle
    v_success := test_utils.exec_sql(
        'INSERT INTO Mutuelles (CodeMutuelle, Libelle, CodeAdresse, Email)
        VALUES ('TST', 'Test Mutuelle', 9999, 'test@mutuelle.com')'
    );

    -- Insertion d'un patient
    v_success := test_utils.exec_sql(
        'INSERT INTO Patients (NrSIS, Nom, Prenom, Sexe, CodeMutuelle, CodeAdresse)
        VALUES ('TEST0000001', 'Allergique', 'Patient', 'M', 'TST', 9999)'
    );

    -- Insertion d'une substance
    v_success := test_utils.exec_sql(
        'INSERT INTO Substances (NrSubstance, Nom) VALUES ('S001', 'Penicilline')'
    );

    -- Insertion d'une autre substance
    v_success := test_utils.exec_sql(
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
'INSERT INTO Substances (NrSubstance, Nom) VALUES ('S002', 'Aspirine')'
);

-- Insertion d'un groupe
v_success := test_utils.exec_sql(
    'INSERT INTO Groupes (NrGroupe, Libelle) VALUES (99, 'Groupe Test')'
);

-- Insertion d'un fabricant
v_success := test_utils.exec_sql(
    'INSERT INTO Fabricants (NrFabricant, Nom, CPays)
    VALUES (99, 'Fabricant Test', 'TE')'
);

-- Insertion d'un médicament contenant de la pénicilline
v_success := test_utils.exec_sql(
    'INSERT INTO Medicaments (NrMedicament, Denomination, NrGroupe, NrFabricant)
    VALUES ('MED001', 'Amoxicilline', 99, 99)'
);

-- Insertion de la composition du médicament
v_success := test_utils.exec_sql(
    'INSERT INTO Composition (NrMedicament, NrSubstance, Quantite, Unite)
    VALUES ('MED001', 'S001', 500, 'mg')'
);

-- Insertion d'un médicament sans pénicilline
v_success := test_utils.exec_sql(
    'INSERT INTO Medicaments (NrMedicament, Denomination, NrGroupe, NrFabricant)
    VALUES ('MED002', 'Paracetamol', 99, 99)'
);

-- Insertion de la composition du médicament
v_success := test_utils.exec_sql(
    'INSERT INTO Composition (NrMedicament, NrSubstance, Quantite, Unite)
    VALUES ('MED002', 'S002', 500, 'mg')'
);

-- Enregistrement d'une allergie à la pénicilline
v_success := test_utils.exec_sql(
    'INSERT INTO Allergies (NrSIS, NrSubstance)
    VALUES ('TEST0000001', 'S001')'
);
test_utils.print_test_result('Enregistrement d'une allergie', v_success);

-- Test 1: Prescription d'un médicament contenant une substance allergène
-- Ce test doit échouer à cause du déclencheur
v_success := test_utils.exec_sql(
    'INSERT INTO Prescriptions (NrPrescription, NrSIS, NrMedicament, DatePrescription)
    VALUES ('PRES001', 'TEST0000001', 'MED001', SYSDATE)'
);
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
test_utils.print_test_result('Prescription d'un médicament allergène (doit échouer)', NOT
v_success);

-- Test 2: Prescription d'un médicament ne contenant pas de substance allergène
-- Ce test doit réussir
v_success := test_utils.exec_sql(
    'INSERT INTO Prescriptions (NrPrescription, NrSIS, NrMedicament, DatePrescription)
    VALUES (''PRES002'', ''TEST0000001'', ''MED002'', SYSDATE)'
);
test_utils.print_test_result('Prescription d'un médicament non allergène', v_success);

-- Nettoyage
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Prescriptions WHERE NrPrescription LIKE ''PRES%''';
EXCEPTION WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Allergies WHERE NrSIS = ''TEST0000001'''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Composition WHERE NrMedicament IN (''MED001'',
''MED002'')'; EXCEPTION WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Medicaments WHERE NrMedicament IN (''MED001'',
''MED002'')'; EXCEPTION WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Fabricants WHERE NrFabricant = 99'; EXCEPTION WHEN OTHERS
THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Groupes WHERE NrGroupe = 99'; EXCEPTION WHEN OTHERS THEN
NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Substances WHERE NrSubstance IN (''S001'', ''S002'')';
EXCEPTION WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Patients WHERE NrSIS = ''TEST0000001'''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Mutuelles WHERE CodeMutuelle = ''TST'''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Adresses WHERE CodeAdresse = 9999'; EXCEPTION WHEN OTHERS
THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Pays WHERE CodePays = ''TE'''; EXCEPTION WHEN OTHERS THEN
NULL; END;
END;
/
```

A.3.2. Test du déclencheur de vérification de l'âge du médecin

```
-- Test du déclencheur TRG_Verif_Age_Medecin
DECLARE
    v_success BOOLEAN;
    v_date_naissance DATE;
    v_date_diplome DATE;
BEGIN
    test_utils.print_test_header('Déclencheur TRG_Verif_Age_Medecin');

    -- Préparation des données de test
    -- Insertion d'une adresse
    v_success := test_utils.exec_sql(
        'INSERT INTO Adresses (CodeAdresse, Adresse, Localite, CodePostal)
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
VALUES (9999, 'Rue de Test', 'TestVille', '12345')'
);

-- Insertion d'une spécialité
v_success := test_utils.exec_sql(
    'INSERT INTO Specialites (Specialite, Libelle) VALUES (''TST'', 'Test Specialite')'
);

-- Test 1: Médecin avec âge valide (diplôme obtenu après 25 ans)
-- Date de naissance: il y a 50 ans
v_date_naissance := ADD_MONTHS(SYSDATE, -12 * 50);
-- Date de diplôme: il y a 20 ans (donc à 30 ans)
v_date_diplome := ADD_MONTHS(SYSDATE, -12 * 20);

v_success := test_utils.exec_sql(
    'INSERT INTO Medecins (NrMedecin, Nom, Prenom, Sexe, DateNaissance, DateDiplome,
CodeAdresse, Specialite)
    VALUES (''TEST0000001'', 'Docteur', 'Valide', 'M',
    TO_DATE('' ' || TO_CHAR(v_date_naissance, 'YYYY-MM-DD') || ' ', 'YYYY-MM-DD'),
    TO_DATE('' ' || TO_CHAR(v_date_diplome, 'YYYY-MM-DD') || ' ', 'YYYY-MM-DD'),
    9999, 'TST')'
);
test_utils.print_test_result('Insertion d'un médecin avec âge valide', v_success);

-- Test 2: Médecin avec âge invalide (diplôme obtenu avant 25 ans)
-- Date de naissance: il y a 40 ans
v_date_naissance := ADD_MONTHS(SYSDATE, -12 * 40);
-- Date de diplôme: il y a 20 ans (donc à 20 ans)
v_date_diplome := ADD_MONTHS(SYSDATE, -12 * 20);

v_success := test_utils.exec_sql(
    'INSERT INTO Medecins (NrMedecin, Nom, Prenom, Sexe, DateNaissance, DateDiplome,
CodeAdresse, Specialite)
    VALUES (''TEST0000002'', 'Docteur', 'Invalide', 'F',
    TO_DATE('' ' || TO_CHAR(v_date_naissance, 'YYYY-MM-DD') || ' ', 'YYYY-MM-DD'),
    TO_DATE('' ' || TO_CHAR(v_date_diplome, 'YYYY-MM-DD') || ' ', 'YYYY-MM-DD'),
    9999, 'TST')'
);
test_utils.print_test_result('Insertion d'un médecin avec âge invalide (doit échouer)', NOT
v_success);

-- Nettoyage
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Medecins WHERE NrMedecin LIKE ''TEST%''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Specialites WHERE Specialite = 'TST''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Adresses WHERE CodeAdresse = 9999; EXCEPTION WHEN OTHERS
THEN NULL; END;
END;
/
```

Laboratoire 1 - Contraintes et Declencheurs SQL

A.3.3. Test du déclencheur de mise à jour du nombre de médecins

```
-- Test du déclencheur TRG_Update_Nbre_Medecins
DECLARE
    v_success BOOLEAN;
    v_nbre_medecins NUMBER;
BEGIN
    test_utils.print_test_header('Déclencheur TRG_Update_Nbre_Medecins');

    -- Préparation des données de test
    -- Insertion d'une adresse pour l'hôpital
    v_success := test_utils.exec_sql(
        'INSERT INTO Adresses (CodeAdresse, Adresse, Localite, CodePostal)
        VALUES (9990, ''Rue de l''Hôpital'', ''TestVille'', ''12345'')'
    );

    -- Insertion d'une adresse pour le médecin
    v_success := test_utils.exec_sql(
        'INSERT INTO Adresses (CodeAdresse, Adresse, Localite, CodePostal)
        VALUES (9991, ''Rue du Médecin'', ''TestVille'', ''12345'')'
    );

    -- Insertion d'une spécialité
    v_success := test_utils.exec_sql(
        'INSERT INTO Specialites (Specialite, Libelle) VALUES (''TST'', ''Test Specialite'')'
    );

    -- Insertion d'un hôpital
    v_success := test_utils.exec_sql(
        'INSERT INTO Hopitaux (NrHopital, Nom, CodeAdresse)
        VALUES (''TEST001'', ''Hôpital Test'', 9990)'
    );

    -- Insertion d'un service
    v_success := test_utils.exec_sql(
        'INSERT INTO Services (NrService, Libelle) VALUES (''S001'', ''Service Test'')'
    );

    -- Insertion d'un médecin
    v_success := test_utils.exec_sql(
        'INSERT INTO Medecins (NrMedecin, Nom, Prenom, Sexe, CodeAdresse, Specialite)
        VALUES (''TEST0000001'', ''Docteur'', ''Test'', ''M'', 9991, ''TST'')'
    );

    -- Insertion d'un lien hôpital-service avec 0 médecins initialement
    v_success := test_utils.exec_sql(
        'INSERT INTO HopitauxServices (NrHopital, NrService, ChefService, NbreMedecins)
        VALUES (''TEST001'', ''S001'', ''TEST0000001'', 0)'
    );

    -- Vérification du nombre initial de médecins
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
EXECUTE IMMEDIATE
    'SELECT NbreMedecins FROM HopitauxServices WHERE NrHopital = 'TEST001' AND NrService =
'S001''
    INTO v_nbre_medecins;

test_utils.print_test_result('Vérification du nombre initial de médecins = 0',
    v_nbre_medecins = 0,
    'Valeur obtenue: ' || v_nbre_medecins);

-- Test 1: Ajout d'un médecin au service
v_success := test_utils.exec_sql(
    'INSERT INTO MedecinsServices (NrHopital, NrService, NrMedecin)
    VALUES ('TEST001', 'S001', 'TEST0000001')'
);
test_utils.print_test_result('Ajout d'un médecin au service', v_success);

-- Vérification du nombre de médecins après ajout
EXECUTE IMMEDIATE
    'SELECT NbreMedecins FROM HopitauxServices WHERE NrHopital = 'TEST001' AND NrService =
'S001''
    INTO v_nbre_medecins;

test_utils.print_test_result('Vérification du nombre de médecins après ajout = 1',
    v_nbre_medecins = 1,
    'Valeur obtenue: ' || v_nbre_medecins);

-- Test 2: Ajout d'un deuxième médecin
v_success := test_utils.exec_sql(
    'INSERT INTO Medecins (NrMedecin, Nom, Prenom, Sexe, CodeAdresse, Specialite)
    VALUES ('TEST0000002', 'Docteur', 'Test2', 'F', 9991, 'TST')'
);

v_success := test_utils.exec_sql(
    'INSERT INTO MedecinsServices (NrHopital, NrService, NrMedecin)
    VALUES ('TEST001', 'S001', 'TEST0000002')'
);
test_utils.print_test_result('Ajout d'un deuxième médecin au service', v_success);

-- Vérification du nombre de médecins après ajout du deuxième
EXECUTE IMMEDIATE
    'SELECT NbreMedecins FROM HopitauxServices WHERE NrHopital = 'TEST001' AND NrService =
'S001''
    INTO v_nbre_medecins;

test_utils.print_test_result('Vérification du nombre de médecins après ajout du deuxième = 2',
    v_nbre_medecins = 2,
    'Valeur obtenue: ' || v_nbre_medecins);

-- Test 3: Suppression d'un médecin
v_success := test_utils.exec_sql(
```

Laboratoire 1 - Contraintes et Declencheurs SQL

```
'DELETE FROM MedecinsServices WHERE NrHopital = 'TEST001' AND NrService = 'S001' AND
NrMedecin = 'TEST0000001'
);
test_utils.print_test_result('Suppression d'un médecin du service', v_success);

-- Vérification du nombre de médecins après suppression
EXECUTE IMMEDIATE
'SELECT NbreMedecins FROM HopitauxServices WHERE NrHopital = 'TEST001' AND NrService =
'S001'
INTO v_nbre_medecins;

test_utils.print_test_result('Vérification du nombre de médecins après suppression = 1',
                             v_nbre_medecins = 1,
                             'Valeur obtenue: ' || v_nbre_medecins);

-- Nettoyage
BEGIN EXECUTE IMMEDIATE 'DELETE FROM MedecinsServices WHERE NrHopital = 'TEST001'';
EXCEPTION WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM HopitauxServices WHERE NrHopital = 'TEST001'';
EXCEPTION WHEN OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Medecins WHERE NrMedecin LIKE 'TEST%''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Services WHERE NrService = 'S001''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Hopitaux WHERE NrHopital = 'TEST001''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Specialites WHERE Specialite = 'TST''; EXCEPTION WHEN
OTHERS THEN NULL; END;
BEGIN EXECUTE IMMEDIATE 'DELETE FROM Adresses WHERE CodeAdresse IN (9990, 9991)'; EXCEPTION
WHEN OTHERS THEN NULL; END;
END;
/
```